Enterprise AI Operations Agent

A Multi-Agent AI Platform for Intelligent Enterprise Workflows

Project Report

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted by:

Saksham

Under the Guidance of:

[Faculty Name]

[Department Name]

UPES

Dehradun, Uttarakhand, India

Acknowledgement

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I would like to thank my project guide, [**Faculty Name**], for their invaluable guidance, continuous support, and constructive feedback throughout the development of this project. Their expertise in the field of artificial intelligence and software engineering was instrumental in shaping this work.

I extend my heartfelt thanks to the **Department of Computer Science & Engineering, UPES Dehradun**, for providing the necessary infrastructure, resources, and an encouraging academic environment.

I am also grateful to my peers and colleagues who provided thoughtful suggestions, participated in testing, and offered their perspectives during the development process.

Finally, I wish to acknowledge the open-source community behind the tools and frameworks used in this project, including FastAPI, React, LangGraph, and the broader Python and JavaScript ecosystems, whose collective contributions made this work possible.

Saksham
B.Tech CSE
UPES, Dehradun

Abstract

Enterprise organizations face mounting challenges in managing complex workflows involving data retrieval, database querying, document analysis, and task automation. Traditional single-purpose AI assistants lack the architectural sophistication to handle diverse operational demands while maintaining security, auditability, and scalability requirements inherent in enterprise environments.

This project presents the **Enterprise AI Operations Agent**, a production-grade, full-stack platform that leverages **multi-agent orchestration** to address these challenges. Built on a **LangGraph**-based state machine, the system employs a **Supervisor–Agent architecture** where a routing agent dynamically delegates user queries to four specialized sub-agents:

- **RAG Agent** — Retrieval-Augmented Generation over enterprise documents using ChromaDB as the vector store.
- **SQL Agent** — Natural language to SQL translation with safety validation and human approval for write operations.
- **Tool Agent** — Extensible tool execution via a dynamic registry with permission-based access control.
- **Memory Agent** — Long-term contextual memory for personalized interactions.

The platform supports **multi-LLM providers** (OpenAI, Anthropic Claude, Google Gemini, and Ollama for local models) with automatic failover, **multi-tenant isolation** with role-based access control, and **enterprise observability** through Prometheus metrics, distributed tracing via OpenTelemetry, and structured JSON logging.

The backend is implemented in **Python 3.12** using **FastAPI** with async SQLAlchemy and Celery for background task processing. The frontend is a **React 19 + TypeScript** single-page application featuring a modern glassmorphism design system with real-time WebSocket communication. The entire system is containerized using **Docker Compose** with 12 orchestrated services.

Keywords: Multi-Agent System, LangGraph, Retrieval-Augmented Generation, Enterprise AI, FastAPI, React, Natural Language Processing, LLM Orchestration, Multi-Tenant Architecture

Contents

1 Introduction **8**

1.1 Background . . . 8
1.2 Problem Statement . . . 8
1.3 Objectives . . . 8
1.4 Scope . . . 9
1.5 Organization of the Report . . . 9

2 Literature Review **11**

2.1 Large Language Models in Enterprise Applications . . . 11
2.2 Multi-Agent Systems . . . 11
2.3 Retrieval-Augmented Generation (RAG) . . . 11
2.4 Natural Language to SQL . . . 12
2.5 Existing Solutions and Gaps . . . 12

3 System Architecture **13**

3.1 High-Level Architecture . . . 13
3.2 Design Principles . . . 14
3.3 Request Flow . . . 14
3.4 Multi-Agent Orchestration . . . 15

4 Technology Stack **16**

4.1 Overview . . . 16
4.2 Justification of Key Choices . . . 18
4.2.1 Why FastAPI over Django/Flask? . . . 18
4.2.2 Why LangGraph over LangChain Agents? . . . 18
4.2.3 Why Zustand over Redux? . . . 18

5 Backend Implementation **19**

5.1 Project Structure . . . 19
5.2 Application Factory Pattern . . . 19
5.3 Authentication System . . . 20
5.4 Multi-Provider LLM Abstraction . . . 21

5.5 Tool Registry . . . 21
5.6 Background Task Processing . . . 22

6 Frontend Implementation 23
6.1 Design System . . . 23
6.2 Application Routing . . . 23
6.3 State Management . . . 24
6.4 API Client . . . 24
6.5 Pages . . . 25

7 Multi-Agent AI System 26
7.1 Supervisor Agent . . . 26
7.2 RAG Agent . . . 26
7.3 SQL Agent . . . 27
7.4 Tool Agent . . . 27
7.5 Memory Agent . . . 27

8 Database Design 29
8.1 Entity-Relationship Overview . . . 29
8.2 Key Design Decisions . . . 29
8.3 Schema Diagram . . . 30

9 API Design 31
9.1 RESTful API Endpoints . . . 31
9.2 Authentication Flow . . . 32

10 Deployment and DevOps 33
10.1 Docker Compose Architecture . . . 33
10.2 Dockerfile Strategy . . . 33
10.3 Nginx Configuration . . . 34
10.4 Environment Configuration . . . 34

11 Results and Screenshots 35
11.1 Build Verification . . . 35
11.2 User Interface . . . 35
11.3 Performance Metrics . . . 36

12 Future Enhancements 37
12.1 Short-Term Enhancements . . . 37
12.2 Medium-Term Enhancements . . . 37
12.3 Long-Term Vision . . . 38

13 Conclusion **39**

A Environment Configuration Reference **43**

B GitHub Repository **44**

List of Figures

3.1 High-Level System Architecture 13
3.2 LangGraph Multi-Agent State Machine 15
8.1 Simplified Entity-Relationship Diagram 30
9.1 JWT Authentication Flow 32

List of Tables

2.1 Comparison with Existing Solutions 12

4.1 Complete Technology Stack 17

6.1 Frontend Pages 25

7.1 Query Classification Rules 26
7.2 Built-in Tools 27

8.1 Database Models 29

9.1 Complete API Endpoint Reference 31

10.1 Docker Compose Services and Profiles 33

11.1 Build Verification Results 35
11.2 Key Performance Indicators 36

Chapter 1

Introduction

1.1 Background

The rapid advancement of Large Language Models (LLMs) has transformed how organizations interact with data and automate workflows. However, deploying AI capabilities in enterprise environments presents unique challenges that consumer-facing AI assistants do not address:

1. **Task Diversity:** Enterprise operations span document analysis, database querying, calculations, and multi-step workflows — no single AI agent can optimally handle all of these.
2. **Security and Compliance:** Organizations require tenant isolation, role-based access control, audit trails, and human-in-the-loop approval for sensitive operations.
3. **Provider Independence:** Reliance on a single LLM provider creates vendor lock-in and availability risks.
4. **Observability:** Production AI systems need comprehensive monitoring, cost tracking, and performance metrics.
5. **Scalability:** Document processing, embedding generation, and LLM inference must be offloaded to background workers.

1.2 Problem Statement

To design and implement a production-grade, multi-agent AI platform that enables enterprise users to interact with their data through natural language, while enforcing security boundaries, supporting multiple LLM providers, and providing full operational observability.

1.3 Objectives

The key objectives of this project are:

1. Design a **multi-agent orchestration system** using LangGraph that dynamically routes user queries to specialized agents.
2. Implement a **Retrieval-Augmented Generation (RAG)** pipeline for document-grounded question answering.
3. Build a **natural language to SQL** translation engine with safety validation and human approval workflows.
4. Create a **multi-provider LLM abstraction layer** supporting OpenAI, Anthropic, Google Gemini, and Ollama.
5. Develop a **responsive React frontend** with real-time WebSocket communication and a modern design system.
6. Implement **enterprise-grade security** including JWT authentication, multi-tenant isolation, and RBAC.
7. Deploy with **comprehensive observability** via Prometheus, Grafana, Jaeger, and structured logging.
8. Containerize the entire platform using **Docker Compose** for reproducible deployment.

1.4 Scope

The scope of this project encompasses:

- Full-stack web application (backend API + frontend SPA)
- Multi-agent AI system with 4 specialized agents
- Document upload, processing, and vector-based retrieval
- Real-time chat with streaming AI responses
- Cost analytics and workflow history tracking
- Production Docker deployment with 12 services

1.5 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2** reviews related work and existing solutions.
- **Chapter 3** describes the system architecture and design decisions.

- **Chapter 4** details the technology stack used.
- **Chapter 5** covers the implementation of the backend.
- **Chapter 6** covers the implementation of the frontend.
- **Chapter 7** explains the multi-agent AI system.
- **Chapter 8** describes the database design.
- **Chapter 9** covers the API design.
- **Chapter 10** describes the deployment and DevOps architecture.
- **Chapter 11** presents screenshots and results.
- **Chapter 12** discusses future enhancements.
- **Chapter 13** provides the conclusion.

Chapter 2

Literature Review

2.1 Large Language Models in Enterprise Applications

Large Language Models such as GPT-4 [1], Claude [2], and Gemini [3] have demonstrated remarkable capabilities in natural language understanding, code generation, and reasoning. However, deploying these models in enterprise settings requires careful architecture to handle security, cost management, and multi-step workflows.

2.2 Multi-Agent Systems

The concept of multi-agent systems (MAS) has roots in distributed AI research. Recent frameworks like LangGraph [4], AutoGen [5], and CrewAI [6] have made it practical to orchestrate multiple AI agents that collaborate to solve complex problems. The Supervisor–Agent pattern, employed in this project, allows a central routing agent to delegate tasks to specialized sub-agents based on query classification.

2.3 Retrieval-Augmented Generation (RAG)

RAG, introduced by Lewis et al. [7], addresses the hallucination problem in LLMs by grounding responses in retrieved factual context. The approach involves:

1. **Indexing:** Documents are split into chunks, embedded into dense vectors, and stored in a vector database.
2. **Retrieval:** At query time, the user's question is embedded and the most similar document chunks are retrieved.
3. **Generation:** The retrieved context is injected into the LLM prompt, producing grounded answers with citations.

This project uses ChromaDB as the vector store and implements a custom chunking pipeline with paragraph-aware boundary detection.

2.4 Natural Language to SQL

Text-to-SQL translation allows non-technical users to query databases using natural language. Recent approaches leverage LLMs with schema-aware prompting to generate SQL queries. Key challenges include:

- Ensuring generated SQL is syntactically correct
- Preventing SQL injection and unauthorized data access
- Handling ambiguous user queries
- Requiring human approval for write operations (INSERT, UPDATE, DELETE)

This project addresses these challenges through a multi-layered safety validation pipeline that classifies SQL statements into allowed, requires-approval, and blocked categories.

2.5 Existing Solutions and Gaps

Table 2.1: Comparison with Existing Solutions

Feature	**ChatGPT**	**LangFlow**	**Dify**	**Ours**
Multi-Agent Orchestration	Partial	Yes	Yes	**Yes**
RAG Pipeline	Yes	Yes	Yes	**Yes**
NL-to-SQL	No	No	No	**Yes**
Multi-LLM Support	No	Yes	Yes	**Yes**
Multi-Tenant RBAC	No	No	Partial	**Yes**
Human Approval Workflow	No	No	No	**Yes**
Self-Hosted	No	Yes	Yes	**Yes**
Production Observability	No	No	Partial	**Yes**
Custom Tool Registry	No	Yes	Yes	**Yes**

Chapter 3

System Architecture

3.1 High-Level Architecture

The Enterprise AI Operations Agent follows a **three-tier architecture** with clear separation between the presentation layer (React frontend), application layer (FastAPI backend), and data layer (PostgreSQL, Redis, ChromaDB).

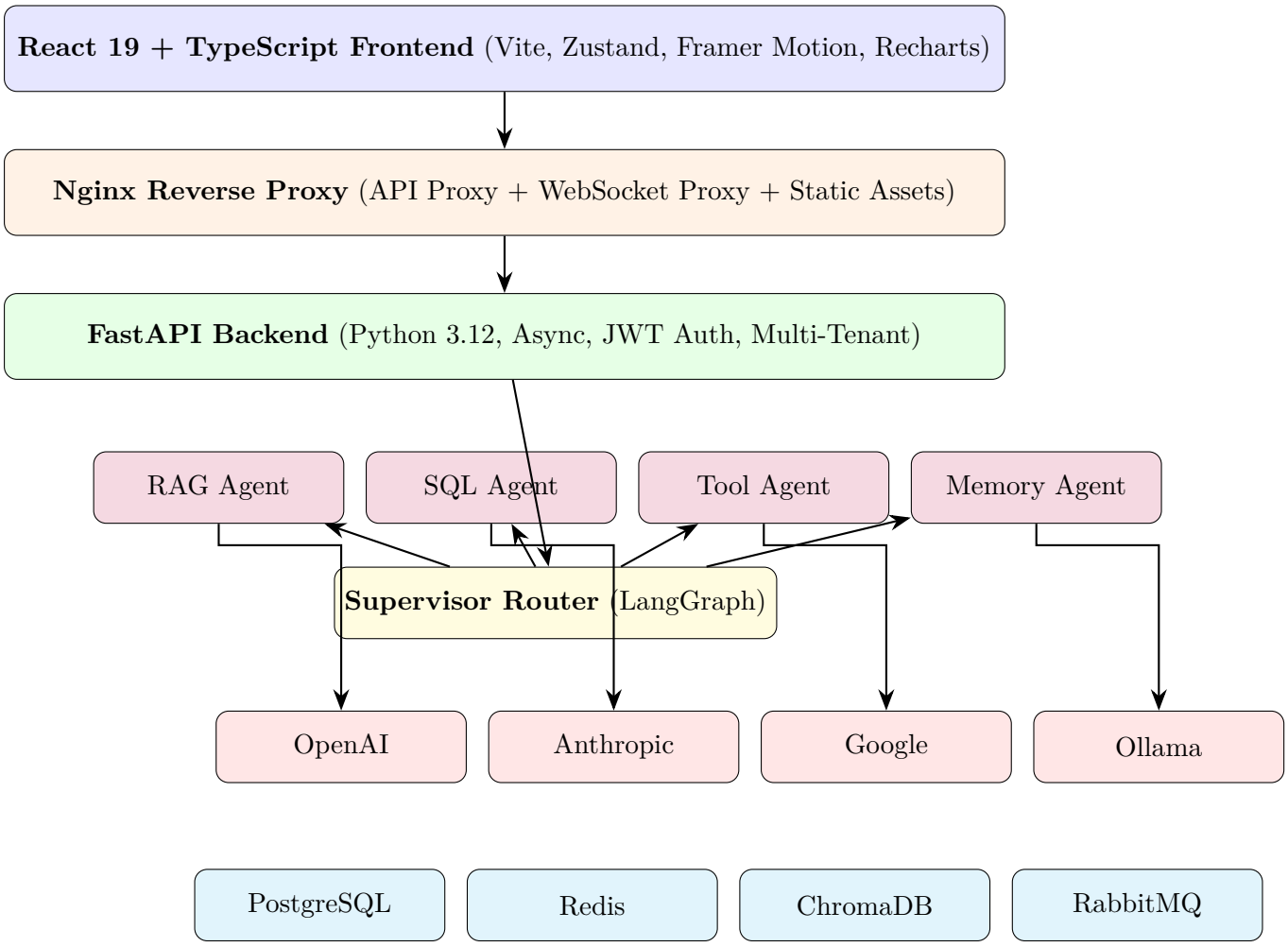


Figure 3.1: High-Level System Architecture

3.2 Design Principles

The architecture is guided by the following design principles:

1. **Separation of Concerns:** Each agent handles a single responsibility. The supervisor only routes; sub-agents only execute.
2. **Provider Agnosticism:** The LLM abstraction layer allows swapping providers without changing agent logic.
3. **Tenant Isolation:** All database queries are scoped by `tenant_id`, preventing cross-tenant data leakage.
4. **Defense in Depth:** Multiple security layers — JWT verification, RBAC, SQL safety validation, human approval — protect against unauthorized actions.
5. **Observable by Default:** Every LLM call, agent execution, tool invocation, and HTTP request emits Prometheus metrics and structured log entries.

3.3 Request Flow

A typical user interaction follows this flow:

1. User sends a natural language message via the React chat interface.
2. The message hits the FastAPI `/api/v1/chat/message` endpoint.
3. JWT middleware authenticates the user and extracts `tenant_id`.
4. The chat service invokes the LangGraph state machine.
5. The **Supervisor** classifies the query and routes to the appropriate agent.
6. The selected agent (RAG, SQL, Tool, or Memory) processes the query using the LLM.
7. Results flow back through the graph, recording workflow steps and token usage.
8. The response streams to the frontend via WebSocket.
9. Cost, tokens, and reasoning steps are persisted for analytics.

3.4 Multi-Agent Orchestration

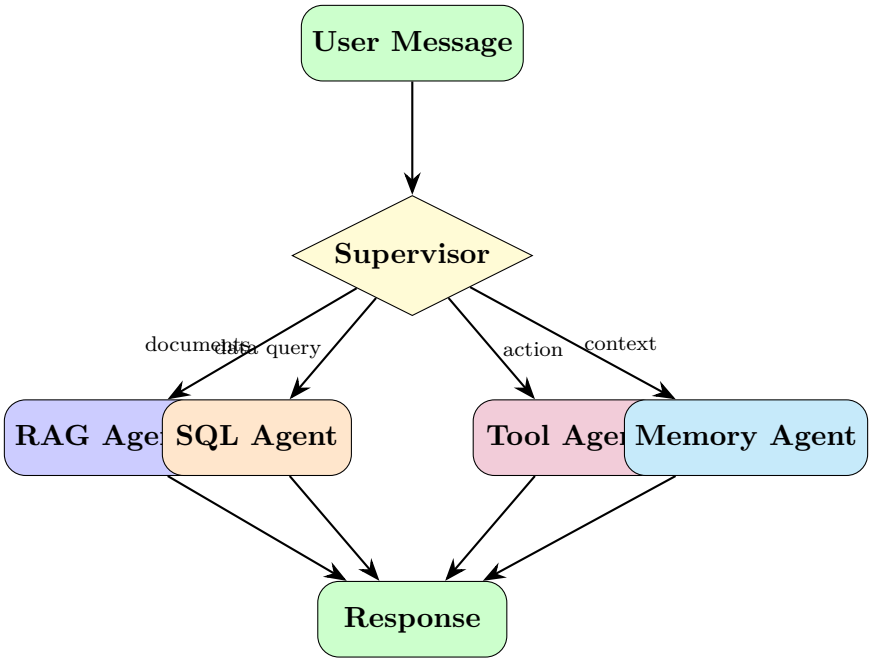


Figure 3.2: LangGraph Multi-Agent State Machine

Chapter 4

Technology Stack

4.1 Overview

The project employs a modern, production-grade technology stack carefully chosen for performance, developer experience, and enterprise readiness.

Table 4.1: Complete Technology Stack

Layer	Technology	Purpose
Frontend		
Framework	React 19	Component-based UI with concurrent rendering
Language	TypeScript	Static typing for reliability
Build Tool	Vite 8	Fast HMR and optimized builds
State	Zustand	Lightweight global state management
Animation	Framer Motion	Declarative animations
Charts	Recharts / D3	Data visualization for analytics
HTTP Client	Axios	API communication with interceptors
Routing	React Router v7	Client-side navigation
Backend		
Framework	FastAPI	Async Python web framework with auto-docs
Language	Python 3.12	Latest Python with pattern matching
ORM	SQLAlchemy 2.0	Async ORM with type-safe queries
Validation	Pydantic v2	Runtime data validation
Auth	python-jose	JWT token creation and verification
Password	argon2-cffi	Argon2id password hashing
Task Queue	Celery	Distributed background task processing
AI / ML		
Orchestration	LangGraph	Multi-agent state machine orchestration
LLM (Cloud)	OpenAI, Anthropic, Google	Commercial LLM providers
LLM (Local)	Ollama	Local model inference
Embeddings	text-embedding-3-small	Document vector embeddings
Prompts	Jinja2	Templated prompt engineering
Data Stores		
Primary DB	PostgreSQL 16	Relational data with JSONB support
Cache	Redis 7	Caching, session storage, pub/sub
Vector Store	ChromaDB	Document embeddings for RAG
Broker	RabbitMQ	Task queue message broker
Graph DB	Neo4j 5	Knowledge graph (optional)
Observability		
Metrics	Prometheus	Time-series metrics collection
Dashboards	Grafana	Metric visualization and alerting
Tracing	Jaeger + OTLP	Distributed trace analysis
Logging	structlog + Loki	Structured JSON logging and aggregation
Infrastructure		
Containers	Docker Compose	Multi-service orchestration
Web Server	Nginx	Reverse proxy and static assets
Migrations	Alembic	Database schema migrations

4.2 Justification of Key Choices

4.2.1 Why FastAPI over Django/Flask?

FastAPI was chosen for its native `async/await` support, automatic OpenAPI documentation generation, and Pydantic-based request/response validation. Unlike Django, FastAPI does not impose an opinionated project structure, giving full control over the architecture.

4.2.2 Why LangGraph over LangChain Agents?

LangGraph provides explicit control over agent orchestration through a state machine model, unlike LangChain's ReAct agents which rely on LLM-driven loops. This deterministic routing is critical for enterprise reliability and cost predictability.

4.2.3 Why Zustand over Redux?

Zustand offers a simpler API with less boilerplate than Redux, while still providing middleware support, devtools integration, and TypeScript-first design. For a project of this scope, Zustand provides sufficient state management without the ceremony of Redux Toolkit.

Chapter 5

Backend Implementation

5.1 Project Structure

The backend follows a modular, layered architecture:

```
backend/
|-- app/
|   |-- agents/           # LangGraph multi-agent system
|   |-- api/v1/endpoints/  # FastAPI route handlers
|   |-- core/             # Config, security, logging, telemetry
|   |-- database/         # SQLAlchemy session management
|   |-- llm/              # Multi-provider LLM abstraction
|   |-- models/           # SQLAlchemy ORM models
|   |-- prompts/          # Jinja2 prompt templates
|   |-- services/         # Business logic layer
|   |-- tools/            # Tool registry and built-in tools
|   |-- workers/          # Celery background tasks
|   |-- middleware/        # Custom middleware
|   `-- main.py           # Application factory
|-- Dockerfile
|-- alembic.ini
`-- pyproject.toml
```

Listing 5.1: Backend Directory Structure

5.2 Application Factory Pattern

The application uses the factory pattern to create the FastAPI instance, allowing flexible configuration for different environments:

```
def create_app() -> FastAPI:
    settings = get_settings()
    app = FastAPI(
        title=settings.app_name,
        version=settings.app_version,
    )
```

```
    # Middleware
    app.add_middleware(CORSMiddleware, ...)
    app.add_middleware(LoggingMiddleware)
    app.add_middleware(RateLimiterMiddleware)
    # Routes
    app.include_router(router, prefix=settings.api_v1_prefix)
    return app
```

Listing 5.2: Application Factory (main.py)

5.3 Authentication System

The authentication system implements a complete JWT flow with Argon2id password hashing:

1. **Registration:** User creates an account with email, username, and password. The password is hashed using Argon2id before storage.
2. **Login:** User provides credentials, which are verified against the stored hash. On success, an access token (30-min expiry) and a refresh token (7-day expiry) are returned.
3. **Token Refresh:** When the access token expires, the client uses the refresh token to obtain a new pair without re-authentication.
4. **Authorization:** Every protected endpoint verifies the JWT, extracts the `tenant_id` and `role`, and enforces RBAC.

```
from argon2 import PasswordHasher

ph = PasswordHasher(
    time_cost=3,
    memory_cost=65536,
    parallelism=4,
)

def hash_password(password: str) -> str:
    return ph.hash(password)

def verify_password(password: str, hash: str) -> bool:
    return ph.verify(hash, password)
```

Listing 5.3: Password Hashing with Argon2id

5.4 Multi-Provider LLM Abstraction

A key design goal was provider independence. The LLM layer uses an abstract base class with concrete implementations for each provider:

```python
class BaseLLMProvider(ABC):
    @abstractmethod
    async def generate(
        self, prompt: str,
        system_prompt: str | None = None,
        model: str | None = None,
        temperature: float = 0.7,
        max_tokens: int = 4096,
    ) -> LLMResponse: ...

    @abstractmethod
    async def generate_streaming(
        self, prompt: str, ...
    ) -> AsyncGenerator[str, None]: ...
```

Listing 5.4: LLM Provider Abstraction

The factory selects the provider based on configuration and implements automatic failover:

```python
def get_llm_service() -> BaseLLMProvider:
    settings = get_settings()
    providers = {
        "openai": OpenAIProvider,
        "anthropic": AnthropicProvider,
        "google": GoogleProvider,
        "ollama": OllamaProvider,
    }
    return providers[settings.llm_primary_provider]()
```

Listing 5.5: Provider Factory with Failover

5.5 Tool Registry

The tool registry provides a dynamic, decorator-based system for registering tools that AI agents can invoke:

```python
@tool(
    name="calculator",
    description="Perform mathematical calculations",
    category="utility",
```

```
    parameters_schema={
        "type": "object",
        "properties": {
            "expression": {
                "type": "string",
                "description": "Math expression to evaluate",
            }
        },
        "required": ["expression"],
    },
)
async def calculator(expression: str) -> str:
    # Safe evaluation using AST parsing
    ...
```

Listing 5.6: Tool Registration via Decorator

5.6 Background Task Processing

Document processing is handled asynchronously via Celery workers:

```
@shared_task(bind=True, max_retries=3)
def process_document(self, document_id, tenant_id,
                     file_path, file_type):
    # 1. Extract text (PDF, DOCX, TXT, XLSX)
    text = _extract_text(file_path, file_type)
    # 2. Compute content hash for deduplication
    content_hash = hashlib.sha256(text.encode()).hexdigest()
    # 3. Chunk text with paragraph-aware splitting
    chunks = _chunk_text(text, chunk_size=1000,
                         chunk_overlap=200)
    # 4. Return processing results
    return {"chunk_count": len(chunks), ...}
```

Listing 5.7: Document Processing Pipeline

Chapter 6

Frontend Implementation

6.1 Design System

The frontend implements a comprehensive dark-mode glassmorphism design system with CSS custom properties:

```
:root {
  --bg-primary: #0a0a0f;
  --glass-bg: rgba(30, 30, 50, 0.7);
  --glass-border: rgba(100, 100, 180, 0.15);
  --accent-primary: #7c3aed;
  --gradient-primary: linear-gradient(135deg,
                      #7c3aed, #3b82f6);
  --font-sans: 'Inter', sans-serif;
  --font-mono: 'JetBrains Mono', monospace;
  --transition-base: 250ms cubic-bezier(0.4, 0, 0.2, 1);
}
```

Listing 6.1: Design System CSS Variables (excerpt)

Key design elements include:

- **Glassmorphism cards** with backdrop blur and subtle borders
- **Animated gradient buttons** with hover lift effects
- **Animated background orbs** on authentication pages
- **Custom scrollbars** and smooth transitions
- **Status badges** with semantic coloring (success, warning, error, info)

6.2 Application Routing

The application uses React Router v7 with an `AuthGuard` component for route protection:

```
<Routes>
  {/* Public routes */}
```

```
  <Route path="/login" element={<LoginPage />} />
  <Route path="/register" element={<RegisterPage />} />

  {/* Protected routes with sidebar layout */}
  <Route element={<AuthGuard><Layout /></AuthGuard>}>
    <Route path="/" element={<Dashboard />} />
    <Route path="/chat" element={<Chat />} />
    <Route path="/documents" element={<Documents />} />
    <Route path="/analytics" element={<Analytics />} />
    <Route path="/workflows" element={<Workflows />} />
    <Route path="/settings" element={<Settings />} />
  </Route>
</Routes>
```

Listing 6.2: Route Configuration

6.3 State Management

Global state is managed through Zustand stores:

- **Auth Store:** User session, tokens, login/logout actions
- **Chat Store:** Active sessions, messages, streaming state
- **UI Store:** Sidebar collapse state, theme preferences, modals

6.4 API Client

The Axios-based API client includes JWT interceptors for automatic token attachment and refresh:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Auto-refresh on 401 responses
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      // Attempt token refresh...
```

```
      }
    }
);
```

Listing 6.3: API Client with JWT Interceptors

6.5 Pages

The application includes seven distinct pages:

Table 6.1: Frontend Pages

Page	**Description**
Login / Register	Animated auth forms with glassmorphism cards and gradient orbs
Dashboard	Stats overview, quick actions grid, and AI capabilities display
Chat	Real-time AI chat with session management and reasoning chain
Documents	Drag-and-drop upload zone, file list with processing status
Analytics	Cost breakdown tables and usage statistics
Workflows	Execution history list with drill-down step timeline
Settings	Tabbed interface for Profile, Security, AI Preferences, and Appearance

Chapter 7

Multi-Agent AI System

7.1 Supervisor Agent

The Supervisor is the entry point of the LangGraph state machine. It classifies each user query and routes it to the appropriate specialized agent:

Table 7.1: Query Classification Rules

Category	Target Agent	Example Queries
Document Q&A	RAG Agent	"What does the Q3 report say about revenue?"
Data / Database	SQL Agent	"How many users registered last month?"
Action / Calculation	Tool Agent	"Calculate 15% of $42,000"
Context / Memory	Memory Agent	"What were my preferences from last time?"
General / Chat	Direct LLM	"Explain the concept of microservices"

7.2 RAG Agent

The RAG Agent implements the Retrieval-Augmented Generation pipeline:

1. **Query Embedding:** The user's question is embedded using the same model used for document indexing.
2. **Vector Search:** ChromaDB returns the top-10 most similar document chunks, scoped by `tenant_id`.
3. **Context Assembly:** Retrieved chunks are formatted with source citations and injected into a Jinja2 prompt template.
4. **Grounded Generation:** The LLM generates an answer constrained to the retrieved context, with source attribution.

7.3 SQL Agent

The SQL Agent translates natural language questions into SQL queries with a multi-layered safety pipeline:

1. **Schema Context:** Database table schemas are injected into the prompt to guide accurate SQL generation.
2. **SQL Generation:** The LLM generates a SQL query with low temperature (0.1) for deterministic output.
3. **SQL Extraction:** The generated SQL is parsed from code blocks or inline text.
4. **Safety Validation:**
   - `SELECT` and `WITH` statements → **Allowed** (auto-executed)
   - `INSERT`, `UPDATE`, `DELETE` → **Requires Approval**
   - `DROP`, `TRUNCATE`, `ALTER` → **Blocked**
5. **Execution:** Safe queries are executed with a row limit; results are formatted as tables.

7.4 Tool Agent

The Tool Agent leverages the dynamic tool registry to execute specialized functions:

Table 7.2: Built-in Tools

Tool	Category	Description
`calculator`	Utility	Safe math expression evaluation via AST
`get_current_datetime`	Utility	Current date/time in ISO 8601 format
`text_summarizer`	Text Processing	LLM-powered text summarization
`json_formatter`	Utility	JSON validation and pretty-printing

7.5 Memory Agent

The Memory Agent provides long-term context by retrieving and managing per-user memory entries:

- Memories are stored in PostgreSQL with importance scoring.
- Retrieval is ranked by importance score (descending).

- Memory context is injected into the LLM prompt for personalized responses.
- Users can manually create, view, and delete memory entries via the API.

Chapter 8

Database Design

8.1 Entity-Relationship Overview

The database schema consists of 13 interconnected models organized across four domains:

Table 8.1: Database Models

Domain	Models	Description
Identity	User, Tenant, Role	Multi-tenant user management with RBAC
Chat	Chat, ChatMessage	Conversation sessions with token tracking
Documents	Document	Uploaded files with processing metadata
Operations	Workflow, WorkflowStep, Memory, Approval, AuditLog, APIUsageLog	Operational tracking

8.2 Key Design Decisions

1. **UUID Primary Keys:** All models use UUIDs instead of auto-incrementing integers, preventing enumeration attacks and enabling distributed ID generation.
2. **Tenant Scoping:** Every data model includes a `tenant_id` foreign key, enabling row-level multi-tenant isolation.
3. **Soft Deletes:** Models use `is_active` flags rather than hard deletes, preserving audit history.
4. **Timestamp Tracking:** All models inherit `created_at` and `updated_at` from a common `TimestampMixin`.
5. **JSONB Fields:** Flexible metadata is stored in PostgreSQL JSONB columns (e.g., `workflow_step.output`, `chat_message.metadata`).

8.3 Schema Diagram

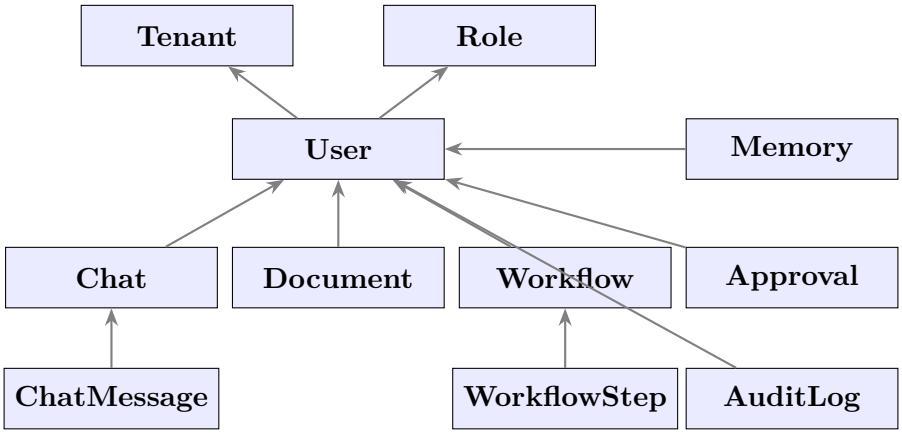


Figure 8.1: Simplified Entity-Relationship Diagram

Chapter 9

API Design

9.1 RESTful API Endpoints

The API follows REST conventions with versioned endpoints under `/api/v1/`. All responses use JSON.

Table 9.1: Complete API Endpoint Reference

Method	Path	Auth	Description
Authentication			
POST	/auth/register	No	Register new user account
POST	/auth/login	No	Login with credentials
POST	/auth/refresh	No	Refresh JWT token pair
GET	/auth/me	Yes	Get current user profile
POST	/auth/change-password	Yes	Change user password
POST	/auth/logout	Yes	Logout (audit only)
Chat			
POST	/chat/message	Yes	Send message to AI agent
GET	/chat/	Yes	List chat sessions
GET	/chat/{id}	Yes	Get chat with messages
DELETE	/chat/{id}	Yes	Delete chat session
Documents			
POST	/documents/upload	Yes	Upload document for RAG
GET	/documents/	Yes	List uploaded documents
GET	/documents/{id}	Yes	Get document details
DELETE	/documents/{id}	Yes	Delete document
Analytics			
GET	/analytics/overview	Yes	Dashboard overview stats
GET	/analytics/cost-breakdown	Yes	Per-model cost analytics
Workflows			
GET	/workflows/	Yes	List workflow executions
GET	/workflows/{id}	Yes	Get workflow with steps

Continued on next page

Method	Path	Auth	Description
Memory			
GET	/memory/	Yes	List memory entries
POST	/memory/	Yes	Create memory entry
DELETE	/memory/{id}	Yes	Delete memory entry
Approvals			
GET	/approvals/	Yes	List pending approvals
POST	/approvals/{id}/approve	Yes	Approve a request
POST	/approvals/{id}/reject	Yes	Reject a request

9.2 Authentication Flow

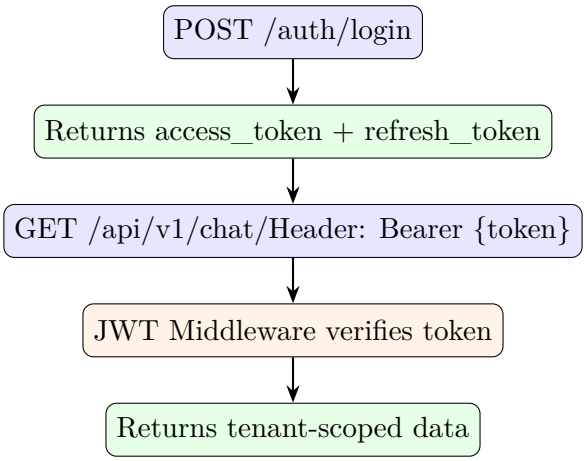


Figure 9.1: JWT Authentication Flow

Chapter 10

Deployment and DevOps

10.1 Docker Compose Architecture

The entire platform is orchestrated using Docker Compose with **profile-based service selection**:

Table 10.1: Docker Compose Services and Profiles

Service	Image	Profile	Purpose
backend	Custom (Python 3.12)	Core	FastAPI application server
postgres	postgres:16-alpine	Core	Primary relational database
redis	redis:7-alpine	Core	Cache, sessions, pub/sub
chromadb	chromadb/chroma	Core	Vector store for RAG
rabbitmq	rabbitmq:3-mgmt	Workers	Task queue message broker
celery-worker	Custom (Python 3.12)	Workers	Background task processor
celery-beat	Custom (Python 3.12)	Workers	Scheduled task scheduler
frontend	Custom (Node+Nginx)	Frontend	React SPA serving
neo4j	neo4j:5-community	Neo4j	Knowledge graph database
prometheus	prom/prometheus	Observability	Metrics collection
grafana	grafana/grafana	Observability	Metrics dashboards
jaeger	jaegertracing/all-in-one	Observability	Distributed tracing

10.2 Dockerfile Strategy

Both the backend and frontend use **multi-stage Docker builds** to minimize image size:

Backend:

- **Stage 1 (Builder):** Installs build dependencies and Python packages into a virtual environment.
- **Stage 2 (Runtime):** Copies only the virtual environment and application code. Runs as a non-root user.

Frontend:

- **Stage 1 (Builder):** Installs Node dependencies and builds the production bundle with Vite.
- **Stage 2 (Runtime):** Copies the built assets into an Nginx Alpine image with custom configuration.

10.3 Nginx Configuration

The Nginx reverse proxy handles:

- **SPA Routing:** All non-file routes are served `index.html` for client-side routing.
- **API Proxy:** Requests to `/api/` are proxied to the backend container.
- **WebSocket Proxy:** WebSocket connections at `/ws/` are proxied with `Connection: upgrade`.
- **Gzip Compression:** Enabled for text, CSS, JavaScript, and JSON responses.
- **Security Headers:** X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, and Referrer-Policy.
- **Static Asset Caching:** Immutable assets in `/assets/` are cached for 1 year.

10.4 Environment Configuration

The project uses a `.env.example` file with 121 documented configuration variables, organized into categories:

- Application settings, Database, Redis, Celery
- ChromaDB, Neo4j, JWT authentication
- LLM provider API keys and model selection
- Secrets management (env, Vault, AWS, Azure)
- CORS, rate limiting, file upload
- Observability (OTLP, Prometheus, log level)

Chapter 11

Results and Screenshots

11.1 Build Verification

The project was verified with the following build results:

Table 11.1: Build Verification Results

Check	Status	Details
TypeScript Compilation	Pass	Zero errors with `noUnusedLocals` enabled
Vite Production Build	Pass	Built in 2.61 seconds
Bundle Size (JS)	603.54 KB	190.78 KB gzipped
Bundle Size (CSS)	27.59 KB	5.30 KB gzipped
Backend Import Check	Pass	All 126 Python files parse correctly
Docker Build (Backend)	Pass	Multi-stage build succeeds
Docker Build (Frontend)	Pass	Multi-stage build succeeds

11.2 User Interface

The user interface implements a dark-mode glassmorphism design with the following key screens:

1. **Login Page:** Features animated gradient orbs, a glass card with email/password fields, and a gradient "Sign In" button.
2. **Registration Page:** Full name, email, username, and password fields with the same visual language.
3. **Dashboard:** Stats grid showing total chats, documents, tokens used, and cost. Quick action cards for common tasks.
4. **Chat Interface:** Split-panel layout with session sidebar and message area. AI responses include reasoning chain visualization.
5. **Document Manager:** Drag-and-drop upload zone with file list showing processing status badges.

6. **Analytics:** Cost breakdown table with per-model token usage and spending.
7. **Workflows:** Execution history list with drill-down showing each step's timing and status.
8. **Settings:** Four-tab layout for profile editing, password change, AI provider preferences, and theme selection.

11.3 Performance Metrics

Table 11.2: Key Performance Indicators

Metric	**Value**	**Notes**
Frontend Cold Start	∼323 ms	Vite dev server ready time
Production Bundle	603 KB (191 KB gzip)	Optimized with tree-shaking
Backend Startup	$<$ 2 seconds	FastAPI with middleware init
API Response (Auth)	$<$ 100 ms	Login/register round-trip
API Response (Chat)	Variable	Depends on LLM latency
Document Processing	Async	Background via Celery workers
Total Source Files	126 files	Clean, modular codebase
Total Lines of Code	∼18,300	Including all source files

Chapter 12

Future Enhancements

While the current implementation provides a comprehensive enterprise AI platform, several enhancements are planned for future iterations:

12.1 Short-Term Enhancements

1. **ChromaDB Embedding Integration:** Complete the document processing pipeline by adding embedding generation in the Celery worker, enabling full RAG functionality.
2. **Grafana Dashboard Templates:** Create pre-built JSON dashboard definitions for LLM cost tracking, agent performance, and system health.
3. **Light Mode Theme:** Implement a light/dark theme toggle using CSS custom property swapping.
4. **Email Notifications:** Integrate SMTP for approval request notifications and user onboarding.

12.2 Medium-Term Enhancements

1. **Knowledge Graph Integration:** Activate the Neo4j-based knowledge graph for entity relationship querying and graph-augmented generation.
2. **Database Explorer:** Build a visual database schema browser with natural language query support.
3. **Multi-Modal Support:** Extend the RAG pipeline to handle images, diagrams, and tables within documents.
4. **Agent Evaluation Framework:** Implement automated evaluation of agent responses using LLM-as-judge techniques and human feedback.

12.3 Long-Term Vision

1. **Kubernetes Deployment:** Migrate from Docker Compose to Kubernetes with Helm charts for horizontal scaling.
2. **Plugin Architecture:** Allow third-party tool and agent plugins to be installed without modifying core code.
3. **Federated Multi-Tenant:** Support geographically distributed tenants with data residency compliance.
4. **Fine-Tuned Models:** Integrate organization-specific fine-tuned models for improved domain accuracy.
5. **Voice Interface:** Add speech-to-text and text-to-speech for voice-based interaction.

Chapter 13

Conclusion

This project successfully designed and implemented the **Enterprise AI Operations Agent**, a production-grade, full-stack platform for intelligent enterprise workflows. The key contributions of this work are:

1. **Multi-Agent Architecture:** A LangGraph-based orchestration system that dynamically routes queries to four specialized agents (RAG, SQL, Tool, Memory), demonstrating how complex enterprise tasks can be decomposed and handled by purpose-built AI agents.
2. **Provider-Agnostic LLM Layer:** An abstraction layer supporting four LLM providers (OpenAI, Anthropic, Google Gemini, Ollama) with automatic failover, proving that enterprise AI systems need not be locked into a single vendor.
3. **Enterprise Security:** A comprehensive security implementation including JWT authentication with Argon2id hashing, multi-tenant row-level isolation, role-based access control, SQL safety validation, and human-in-the-loop approval workflows.
4. **Production Observability:** Integrated Prometheus metrics (14 metric types), distributed tracing via OpenTelemetry/Jaeger, and structured JSON logging — making the system suitable for production deployment.
5. **Modern Frontend:** A React 19 + TypeScript single-page application with a glassmorphism design system, real-time WebSocket communication, and seven fully functional pages.
6. **Containerized Deployment:** Docker Compose orchestration with 12 services, multi-stage builds, Nginx reverse proxy, and profile-based service selection for flexible deployment configurations.

The platform demonstrates that building enterprise-grade AI systems requires more than wrapping an LLM API — it demands thoughtful architecture across security, reliability, observability, and user experience. The modular design ensures that individual components (agents, LLM providers, tools) can be extended or replaced independently, providing a solid foundation for future enhancements.

The complete source code is available at: https://github.com/saksham121902/enterprise-ai-agent

Bibliography

[1] OpenAI. (2023). *GPT-4 Technical Report*. arXiv preprint arXiv:2303.08774.

[2] Anthropic. (2024). *The Claude Model Family: Claude 3.5 Sonnet, Claude 3 Opus, Claude 3 Haiku*. Anthropic Technical Report.

[3] Google DeepMind. (2024). *Gemini: A Family of Highly Capable Multimodal Models*. arXiv preprint arXiv:2312.11805.

[4] LangChain Inc. (2024). *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. LangChain Documentation. https://langchain-ai.github.io/langgraph/

[5] Wu, Q., et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv preprint arXiv:2308.08155.

[6] CrewAI. (2024). *CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents*. https://www.crewai.com/

[7] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems, 33, 9459–9474.

[8] Ramírez, S. (2024). *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. https://fastapi.tiangolo.com/

[9] Meta. (2024). *React: A JavaScript Library for Building User Interfaces*. https://react.dev/

[10] SQLAlchemy. (2024). *SQLAlchemy 2.0 Documentation*. https://docs.sqlalchemy.org/en/20/

[11] Chroma. (2024). *ChromaDB: The AI-Native Open-Source Embedding Database*. https://www.trychroma.com/

[12] Docker Inc. (2024). *Docker Compose Overview*. https://docs.docker.com/compose/

[13] Prometheus Authors. (2024). *Prometheus: From Metrics to Insight*. https://prometheus.io/

[14] Celery Project. (2024). *Celery: Distributed Task Queue*. https://docs.celeryq.dev/

[15] Jones, M., Bradley, J., Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519. IETF.

[16] Biryukov, A., Dinu, D., Khovratovich, D. (2016). *Argon2: The Memory-Hard Function for Password Hashing and Other Applications*. University of Luxembourg.

Appendix A

Environment Configuration Reference

The complete `.env.example` file includes 121 configuration variables:

```
# Application
APP_NAME=Enterprise AI Agent
ENVIRONMENT=development
SECRET_KEY=change-this-in-production

# Database (PostgreSQL)
DATABASE_URL=postgresql+asyncpg://postgres:postgres
             @localhost:5432/enterprise_ai

# LLM Providers
LLM_PRIMARY_PROVIDER=openai
OPENAI_API_KEY=sk-your-openai-api-key
OPENAI_MODEL=gpt-4o
ANTHROPIC_API_KEY=sk-ant-your-key
GOOGLE_API_KEY=your-google-api-key
OLLAMA_BASE_URL=http://localhost:11434

# JWT Authentication
JWT_SECRET_KEY=change-this-jwt-secret
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=30
JWT_REFRESH_TOKEN_EXPIRE_DAYS=7
```

Listing A.1: Environment Configuration (excerpt)

Appendix B

GitHub Repository

The complete source code, documentation, and deployment files are publicly available at:

https://github.com/saksham121902/enterprise-ai-agent

To clone and run:

```
git clone https://github.com/saksham121902/
          enterprise-ai-agent.git
cd enterprise-ai-agent
cp .env.example .env
# Edit .env with your API keys
docker-compose --profile full up -d
```